

Vehicle Detection in Satellite Images

GNR602

Tushar Singha Roy (23B1233)
Abhishek Kushwah (210100004)
Nasiruddin Kabir (23B0041)

May 3, 2025

1. Introduction

The ability to spot vehicles in satellite images is becoming more important for areas like traffic monitoring, emergency response, and security. Satellite images give us a wide, top-down view of large areas, which is very useful - but they also come with some challenges:

- The images are often low in detail (low resolution)
- Lighting and shadows can vary a lot between images
- There's a lot of background clutter like buildings, trees, and roads

Because of these issues, regular object detection techniques don't always work well. In this project, we use a lightweight version of the YOLO model that can detect objects quickly and accurately.

2. Dataset and Preprocessing

We sourced satellite images containing vehicles from online repositories and open datasets. Each image includes bounding box annotations marking vehicle locations. The dataset was split as follows:

- **Training set:** 3206 images
- **Validation set:** 190 images
- **Test set:** 206 images

2.1. Data Augmentation

To prevent overfitting and make the model robust to real-world conditions, we applied the following augmentations to the training data:

- **Brightness Variation:** 0.5% adjustment to simulate different daylight and weather lighting conditions.
- **Exposure Modification:** 0.5% changes to emulate exposure differences across various satellites or camera sensors.
- **Gaussian Blur:** 0.9-pixel blur to replicate sensor noise, motion blur, or image compression artifacts.
- **Random Noise:** 0.3% of pixels randomly altered to mimic environmental noise and digital distortion.
- **Dynamic Cropping and Resizing:** Performed to evaluate how well the model generalizes to different input sizes and aspect ratios, simulating real-world variability in satellite resolutions.

These augmentations help the model generalize better across unseen lighting, terrain, and satellite sensor types.

3. Model and Methodology

3.1. YOLOv11n Overview

YOLO (You Only Look Once) is an end-to-end object detection framework known for its speed and efficiency. YOLOv11n is a newer, compact variant optimized for fast inference with lower computational resources.

Advantages:

- Real-time inference capability
- Compact architecture for edge or low-power devices
- Single-stage detection with bounding box and class prediction in one pass

3.2. Model Configuration

- **Image Size:** 640x640
- **Batch Size:** 16
- **Epochs:** 100 (default: 100)
- **Optimizer:** Stochastic Gradient Descent
- **Momentum:** 0.937

- **Weight Decay:** 0.0005
- **Learning Rate:** Starts at 0.01
- **Loss Function:**
 - CIoU loss for bounding box regression
 - Binary Cross Entropy (BCE) with logits for objectness and class prediction
- **Early Stopping:** Patience = 10

4. Training Iterations

4.1. Iteration 1: Training (100 Epochs)

We trained the model for 100 epochs.

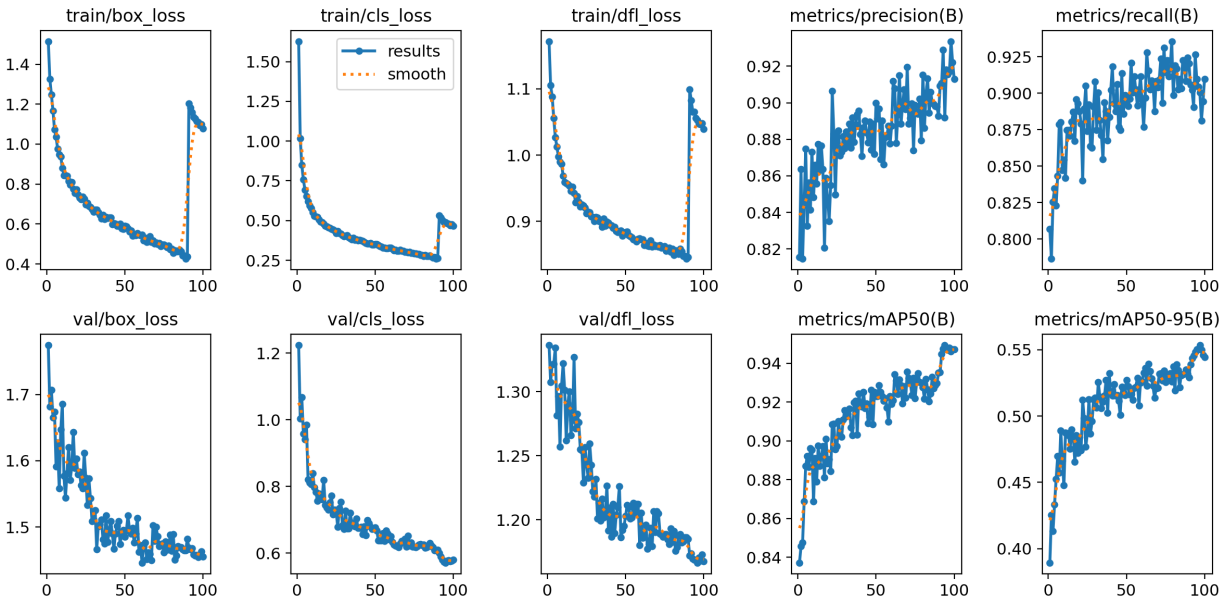
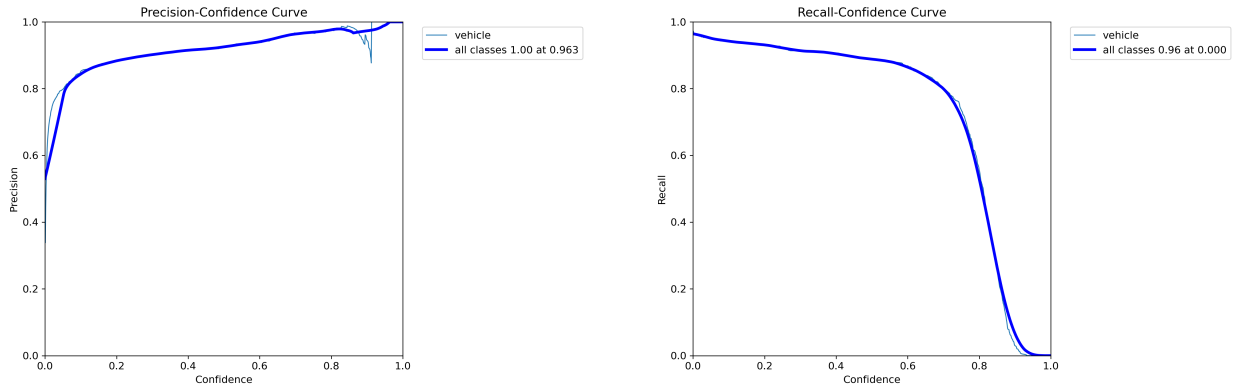


Figure 1: Results



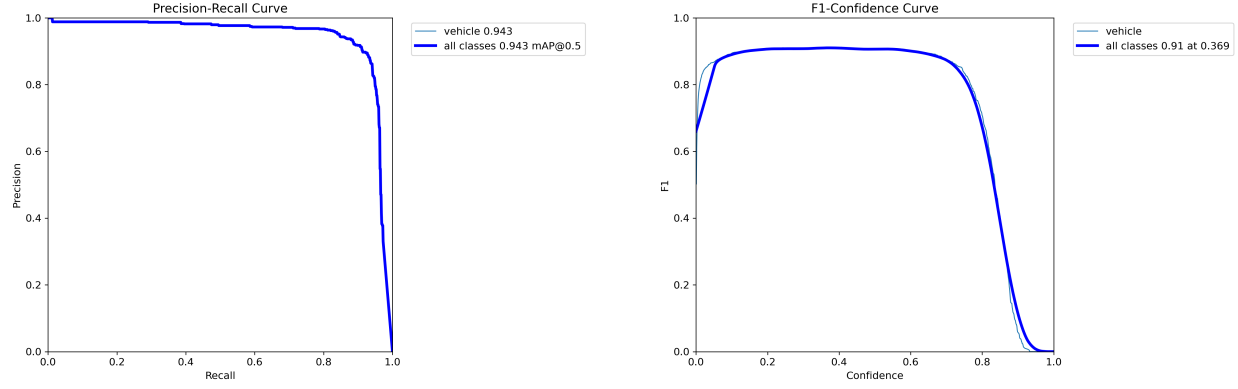


Figure 2: Precision, Recall and F1-Score



Figure 3: Train (Left) and Validation (Right) Images

4.2. Iteration 2: Early Stopping (49 Epochs)

We retrained the model with early stopping turned ON as losses increased after 90 epochs suddenly. To prevent overfitting, early stopping was triggered after 49 epochs.

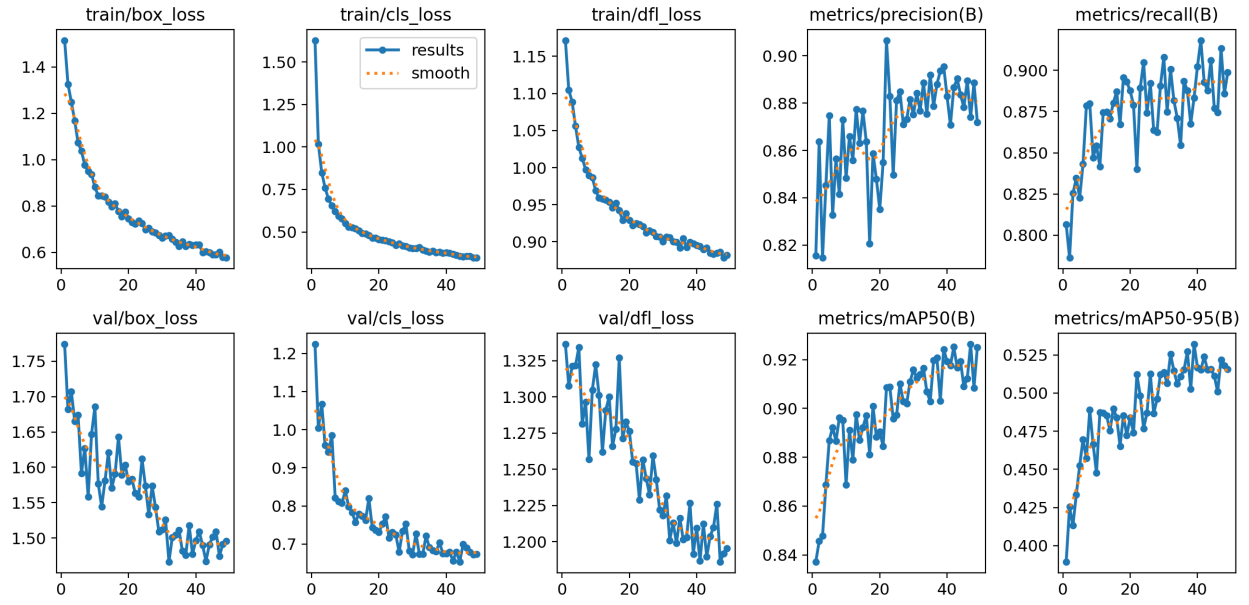


Figure 4: Results

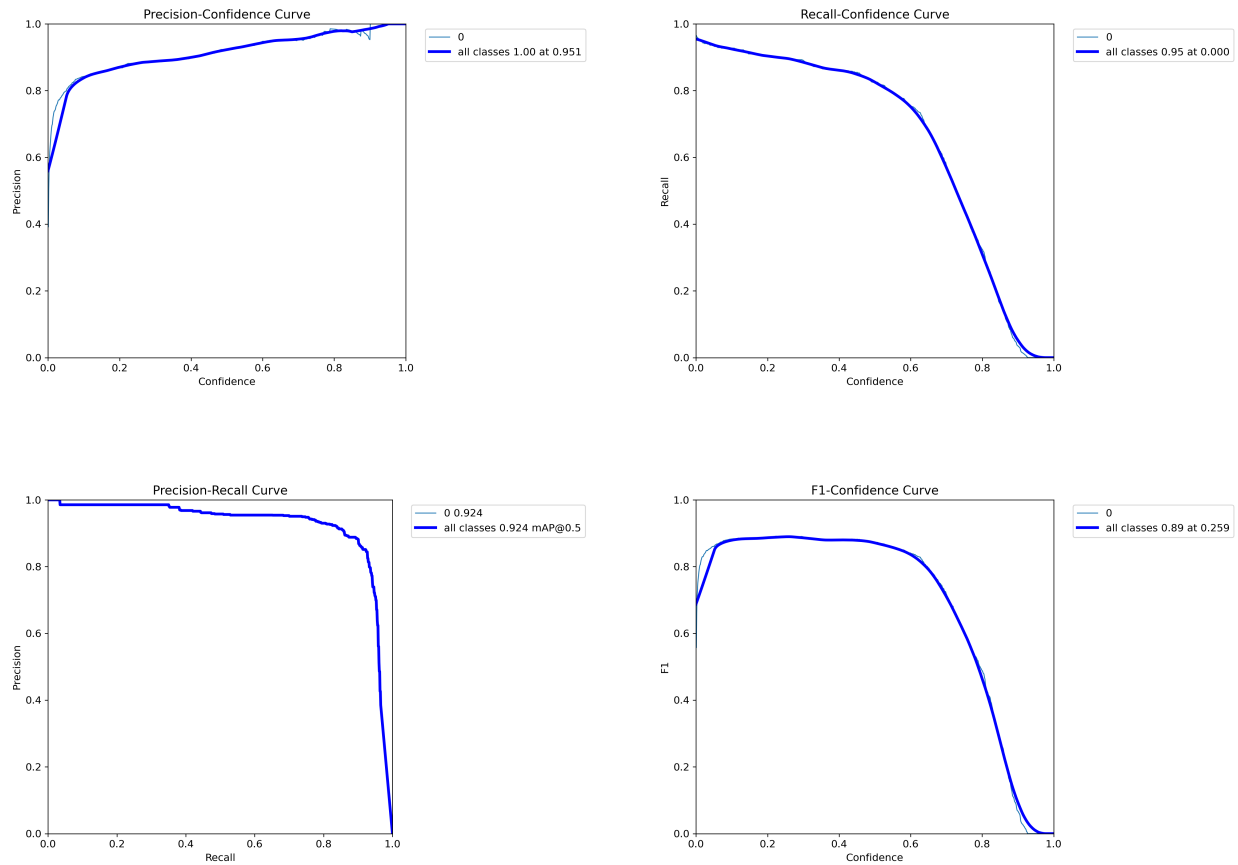


Figure 5: Precision, Recall and F1-Score



Figure 6: Train (Left) and Validation (Right) Images

5. Testing & Evaluation

After completing training, both models from Iteration 1 (100 epochs) and Iteration 2 (early stopped at 49 epochs) were evaluated on a separate test set containing 206 images.

5.1. Testing Setup

Each model was loaded using the Ultralytics YOLO interface and tested using consistent image sizes of 256x256. Predictions were saved, and evaluation metrics such as precision, recall, and mean Average Precision (mAP) were extracted using built-in evaluation utilities.

5.2. Evaluation Metrics Comparison

The table below compares the performance of the two models on the modified test set:

Metric	Iteration 1 (100 Epochs)	Iteration 2 (49 Epochs)
Precision (P)	0.917	0.886
Recall (R)	0.854	0.858
mAP@0.5	0.922	0.903
mAP@0.5:0.95	0.549	0.526
Inference Time (per image)	213.9 ms	211.1 ms

Table 1: Test performance comparison of both trained models

5.3. Final Model Selection

While both models performed reasonably well, the model from Iteration 1 (trained for 100 epochs) consistently achieved higher accuracy across key metrics. Therefore, this version was selected as the final deployed model for vehicle detection in satellite images.

6. Implementation Details

6.1. Training Code

```
from ultralytics import YOLO

model = YOLO('yolov11n.pt')

results = model.train(
    data='datasets/data.yaml',
    epochs=100,
    imgsz=640,
    batch=16,
    name='model_v11',
    device=0,
    verbose=True,
    val=True,
    plots=True,
    save=True,
    save_period=10,
    workers=6,
    exist_ok=True,
    cache=True,
    patience=10,
    optimizer='SGD',
    lr0=0.01,
    momentum=0.937,
    weight_decay=0.0005
)
```

6.2. Testing Code

```
from ultralytics import YOLO

model = YOLO('yolov11n/weights/best.pt')

metrics = model.val(
    data='datasets/data.yaml',
    split='test',
    imgsz=640,
```



```
    batch=16,  
    device=0,  
    save_json=True,  
    verbose=True,  
    plots=True  
)
```

6.3. Deploying Code

```
image_path = 'img.jpg'  
image = cv2.imread(image_path)  
  
results = model.predict(source=image, save=False, conf=0.25)  
  
for r in results:  
    for box in r.bboxes:  
        x1, y1, x2, y2 = map(int, box.xyxy[0])  
        conf = float(box.conf[0])  
        cls = int(box.cls[0])  
        label = f'{model.names[cls]} {conf:.2f}'  
  
        cv2.rectangle(image, (x1, y1), (x2, y2), (0, 255, 0), 2)  
        cv2.putText(image, label, (x1, y1 - 10), cv2.  
            FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)  
  
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)  
  
plt.figure(figsize=(10, 8))  
plt.imshow(image_rgb)  
plt.axis('off')  
plt.title('Detection Result')  
plt.show()
```

7. Conclusion

We successfully trained a lightweight YOLOv11n model for detecting vehicles in satellite images. With robust augmentation and resolution adaptation strategies, the model generalizes well to new conditions. The project demonstrates the feasibility of applying real-time detection techniques in overhead imagery domains with limited hardware.